

Diogo Scherrer
Gabriel Furtunato
Gabriel Xavier

Controle de Estoque de Livros Didáticos

Controle de Livros - Senai

Limeira – SP
2026

Sumário

1. Visão Geral do Sistema	3
1.1 Contexto	3
1.2 Objetivo	3
1.3 Escopo	3
2. Stakeholders e Atores	4
2.1 Stakeholders	4
2.2 Atores (usuários do sistema)	4
3. Requisitos Funcionais (RF)	4
RF1 - Autenticação (Login)	4
RF2 - Controle de Perfil e Permissões	5
RF3 - Cadastro de Livros (Catálogo)	5
RF4 - Listagem de Livros	5
RF5 - Entrada de Estoque (Abastecimento)	6
RF6 - Saída de Estoque (Baixa Manual)	6
RF7 - Consulta de Saldo por Livro	6
RF8 - Monitoramento de Baixo Estoque	7
RF9 - Histórico de Movimentações	7
RF10 - Logout	7
4. Regras de Negócio (RN)	8
RN1 - Estoque não pode ficar negativo	8
RN2 - Quantidade deve ser positiva	8
RN3 - ISBN deve ser único	8
RN4 - Movimentação deve registrar autor e data	8
RN5 - Operações críticas devem ser atômicas	8
RN6 - Nível mínimo de estoque	8
5. Requisitos Não Funcionais (RNF)	8
RNF1 - Padrão de API	8
RNF2 - Segurança	8
RNF3 - Qualidade de Código	9
RNF4 - Persistência e Integridade	9
RNF5 - Performance mínima	9
RNF6 - Versionamento e rastreabilidade	9
RNF7 - Testabilidade	9
6. Análise MoSCoW	10
7. Diagrama de Classe	10
8. Diagrama de Sequência	10
9. Diagrama do Banco de Dados	11
10. Metodologia de Desenvolvimento	13
10.1 Metodologia Ágil Utilizada	13
10.2 Organização das Sprints	13
10.3 Ferramenta de Gestão	14

1. Visão Geral do Sistema

1.1 Contexto

O Senai envia periodicamente grandes remessas de livros para as unidades de ensino. Esses livros ficam armazenados no almoxarifado e são retirados pelos instrutores para serem distribuídos em sala de aula conforme a necessidade. O problema atual é a falta de controle sobre o saldo remanescente. O almoxarifado sabe quantos livros chegaram, mas perde o controle de quantos saíram. Frequentemente, descobre-se que o estoque de um título importante zerou apenas no momento em que uma turma precisa dele, gerando atrasos no aprendizado.

1.2 Objetivo

Desenvolver o Back-End do SenaiStock, uma API RESTful construída com Laravel e MySQL, com o propósito de garantir o controle preciso e a rastreabilidade do estoque de livros didáticos do SENAI.

O sistema deve assegurar que o saldo de cada título esteja sempre atualizado em tempo real, eliminando as falhas de controle que atualmente geram rupturas de estoque e atrasos no aprendizado.

- Entradas (abastecimento/chegada de livros)
- Saídas (baixa manual/retiradas para turmas)
- Alertas de baixo estoque
- Histórico de Movimentações
- Controle de Acesso

1.3 Escopo

- Definir requisitos funcionais e não funcionais
 - Definir regras de negócio
 - Definir atores/perfis e permissões
 - Definir endpoints esperados (alto nível)
 - Definir critérios de aceitação
-

2. Stakeholders e Atores

2.1 Stakeholders

- **Almoxarifado (SENAI)**: precisa do saldo confiável e rapidez na operação.
- **Coordenação**: precisa acompanhar e garantir reposição.
- **Instrutores**: dependem do material disponível
- **TI / Desenvolvimento**: mantém e implanta o sistema.

2.2 Atores (usuários do sistema)

- **Almoxarife** (operacional): cadastra livros, registra entradas e saídas.
 - **Coordenador** (gestão): consulta relatórios, monitora baixo estoque e pode ajustar/cadastrar.
 - **Sistema/API**: valida regras e controla permissões.
-

3. Requisitos Funcionais (RF)

RF1 - Autenticação (Login)

Descrição: O sistema deve permitir autenticação para garantir acesso apenas a funcionários autorizados.

Entrada: email / usuário e senha.

Saída: token de autenticação e dados básicos do usuário.

Prioridade: Alta.

Critérios de aceitação:

- Deve ser possível realizar login com credenciais válidas.
 - Deve retornar erro e status adequado quando inválidas.
 - Rotas protegidas devem bloquear acesso sem token.
-

RF2 - Controle de Perfil e Permissões

Descrição: O sistema deve controlar o que cada perfil pode fazer.

Perfis: Almoxarife, Coordenador.

Prioridade: Alta.

Regras mínimas:

- Almoxarife: pode cadastrar livro, registrar entrada e saída.
- Coordenador: pode consultar, monitorar baixo estoque e também registrar movimentações.

Critérios de aceitação:

- Um usuário sem permissão não consegue executar rotas restritas.
-

RF3 - Cadastro de Livros (Catálogo)

Descrição: O sistema deve permitir cadastrar livros no catálogo.

Campos: Título, ISBN, Matéria.

Prioridade: Alta.

Critérios de aceitação:

- Não permitir ISBN duplicado.
 - Validar campos obrigatórios.
 - Retornar o livro cadastrado em JSON.
-

RF4 - Listagem de Livros

Descrição: O sistema deve listar livros cadastrados com seus dados e saldo atual.

Prioridade: Alta.

Critérios de aceitação:

- Deve retornar lista em JSON.
 - Deve permitir paginação (recomendado).
 - Deve permitir filtro por título/ISBN/matéria (recomendado).
-

RF5 - Entrada de Estoque (Abastecimento)

Descrição: Registrar chegada de livros e somar ao saldo atual do livro.

Entrada: livro_id (ou ISBN) e quantidade.

Saída: saldo atualizado e registro da movimentação.

Prioridade: Alta.

Critérios de aceitação:

- Quantidade deve ser > 0.
 - Deve registrar “movimentação do tipo ENTRADA”.
 - O saldo deve aumentar corretamente.
-

RF6 - Saída de Estoque (Baixa Manual)

Descrição: Registrar retirada de livros e subtrair do saldo atual.

Entrada: livro_id (ou ISBN), quantidade e observação (ex.: turma/justificativa).

Saída: saldo atualizado e registro da movimentação.

Prioridade: Altíssima (principal do sistema).

Critérios de aceitação:

- Quantidade deve ser > 0.
 - Deve registrar “movimentação do tipo SAÍDA”.
 - Deve exigir observação (recomendado).
 - Não permitir saída se **quantidade > saldo disponível** (ver RN-01).
 - Retornar erro claro “Estoque insuficiente” com status adequado.
-

RF7 - Consulta de Saldo por Livro

Descrição: Permitir consultar o saldo atual de um livro específico.

Prioridade: Média.

Critérios de aceitação:

- Retornar dados do livro + saldo atual.
-

RF8 - Monitoramento de Baixo Estoque

Descrição: Listar livros com estoque abaixo de um nível mínimo configurado.

Entrada: parâmetro mínimo (opcional) ou valor padrão (ex.: 10).

Saída: lista de livros abaixo do mínimo.

Prioridade: Alta.

Critérios de aceitação:

- Se não informar mínimo, usar padrão.
 - Retornar lista em JSON ordenada por menor saldo.
-

RF9 - Histórico de Movimentações

Descrição: Permitir consultar entradas e saídas registradas.

Filtros: por livro, por período, por tipo (entrada/saída), por usuário.

Prioridade: Média/Alta.

Critérios de aceitação:

- Retornar lista paginada.
 - Deve mostrar: tipo, quantidade, data/hora, usuário, observação, livro.
-

RF10 - Logout

Descrição: Encerrar sessão/token.

Prioridade: Média.

Critérios de aceitação:

- Após logout, token não deve acessar rotas protegidas.
-

4. Regras de Negócio (RN)

RN1 - Estoque não pode ficar negativo

Se uma saída solicitar quantidade maior que o saldo atual, a operação deve ser **bloqueada**.

RN2 - Quantidade deve ser positiva

Não permitir entrada/saída com quantidade 0 ou negativa.

RN3 - ISBN deve ser único

Não permitir cadastro de dois livros com o mesmo ISBN.

RN4 - Movimentação deve registrar autor e data

Toda entrada/saída deve gravar **usuário autenticado** e **timestamp**.

RN5 - Operações críticas devem ser atômicas

Entrada/saída devem ser executadas em **transação**, garantindo consistência.

RN6 - Nível mínimo de estoque

O sistema deve usar um nível mínimo padrão (ex.: 10) ou permitir parâmetro na consulta.

5. Requisitos Não Funcionais (RNF)

RNF1 - Padrão de API

- API RESTful
- Respostas em JSON
- Uso correto de status HTTP (200, 201, 400, 401, 403, 404, 409, 422)

RNF2 - Segurança

- Rotas de alteração de estoque protegidas por autenticação
- Senhas armazenadas com hash seguro
- Validação de entrada (evitar SQL injection; padrão Laravel já ajuda)

RNF3 - Qualidade de Código

- Padrões PSR
- Clean Code
- Separação de responsabilidades (Controllers, Services, Requests)

RNF4 - Persistência e Integridade

- MySQL com chaves e restrições
- Índices para ISBN e buscas frequentes

RNF5 - Performance mínima

- Listagens com paginação
- Consultas filtradas e indexadas

RNF6 - Versionamento e rastreabilidade

- Git + GitHub
- Commits descritivos
- Tags/releases (opcional)

RNF7 - Testabilidade

- Rotas testáveis via Insomnia/Postman
 - Evidências de testes (prints/coleção exportada)
-

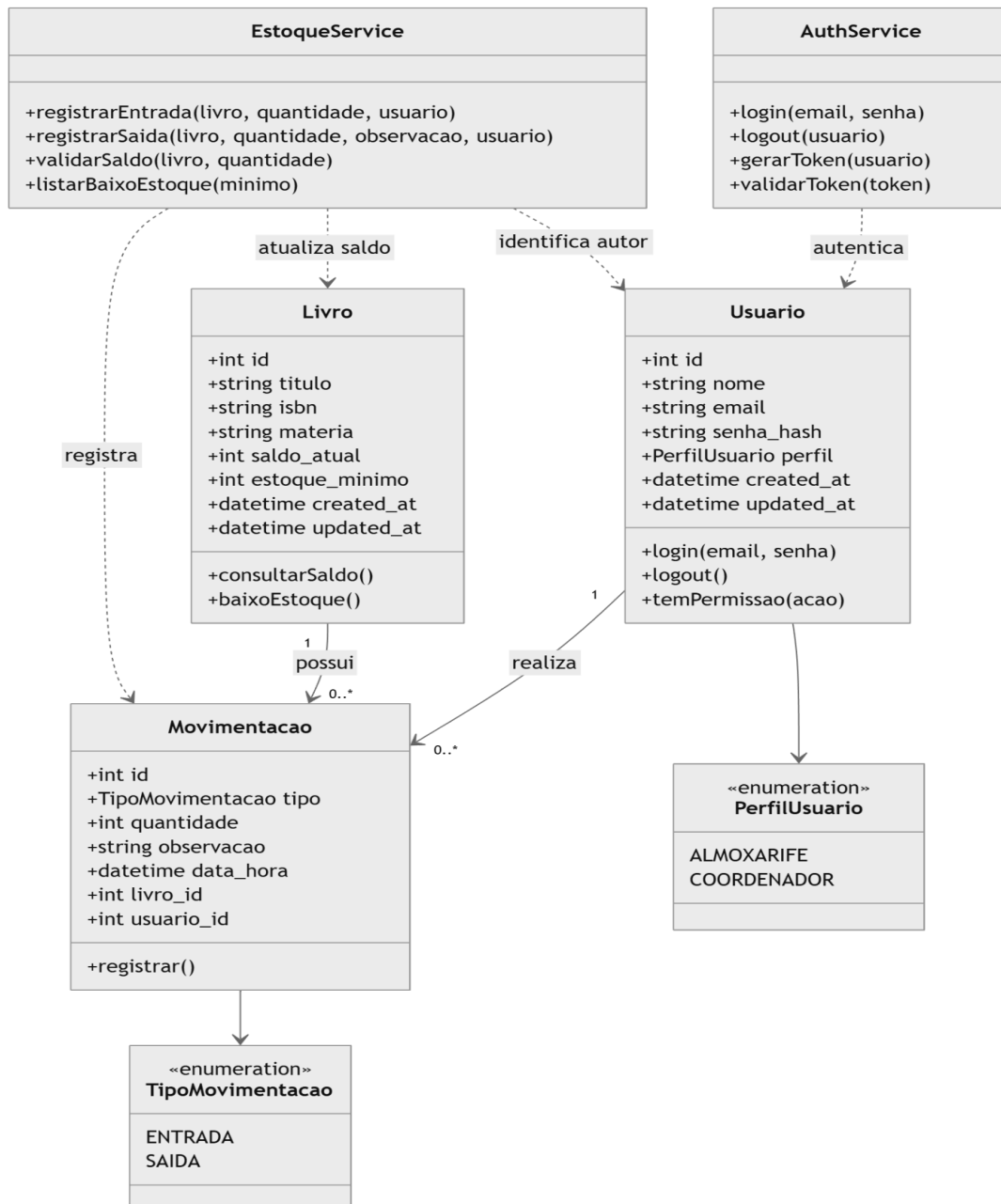
6. Análise MoSCoW

A técnica MoSCoW prioriza os requisitos em quatro categorias. Must Have são obrigatórios para o funcionamento mínimo do sistema: Autenticação, que garante segurança no acesso, e Saída de Estoque, funcionalidade principal que resolve o problema de controle. Should Have são importantes mas não bloqueiam o MVP: Cadastro de Livros e Entrada de Estoque. Could Have são melhorias desejáveis: Baixo Estoque e Listagem de Livros. Won't Have ficam fora do escopo atual: Relatórios Gráficos e Notificações por Email.

Categoria	ID	Requisito	Descrição
 MUST HAVE ▾	RF-01 ▾	Autenticação	Login com email/senha, retornar token
 MUST HAVE ▾	RF-06 ▾	Saída de Estoque	Registrar retirada, validar saldo
 SHOULD HAVE ▾	RF-03 ▾	Cadastro de Livros	Título, ISBN, Matéria (ISBN único)
 SHOULD HAVE ▾	RF-05 ▾	Entrada de Estoque	Registrar abastecimento, somar ao saldo
 COULD HAVE ▾	RF-08 ▾	Baixo Estoque	Listar livros abaixo do mínimo
 COULD HAVE ▾	RF-04 ▾	Listagem de Livros	Listar com saldo atual e filtros
 WON'T HAVE ▾	- ▾	Relatórios Gráficos	Dashboards com gráficos
 WON'T HAVE ▾	- ▾	Notificações por Email	Alertas automáticos

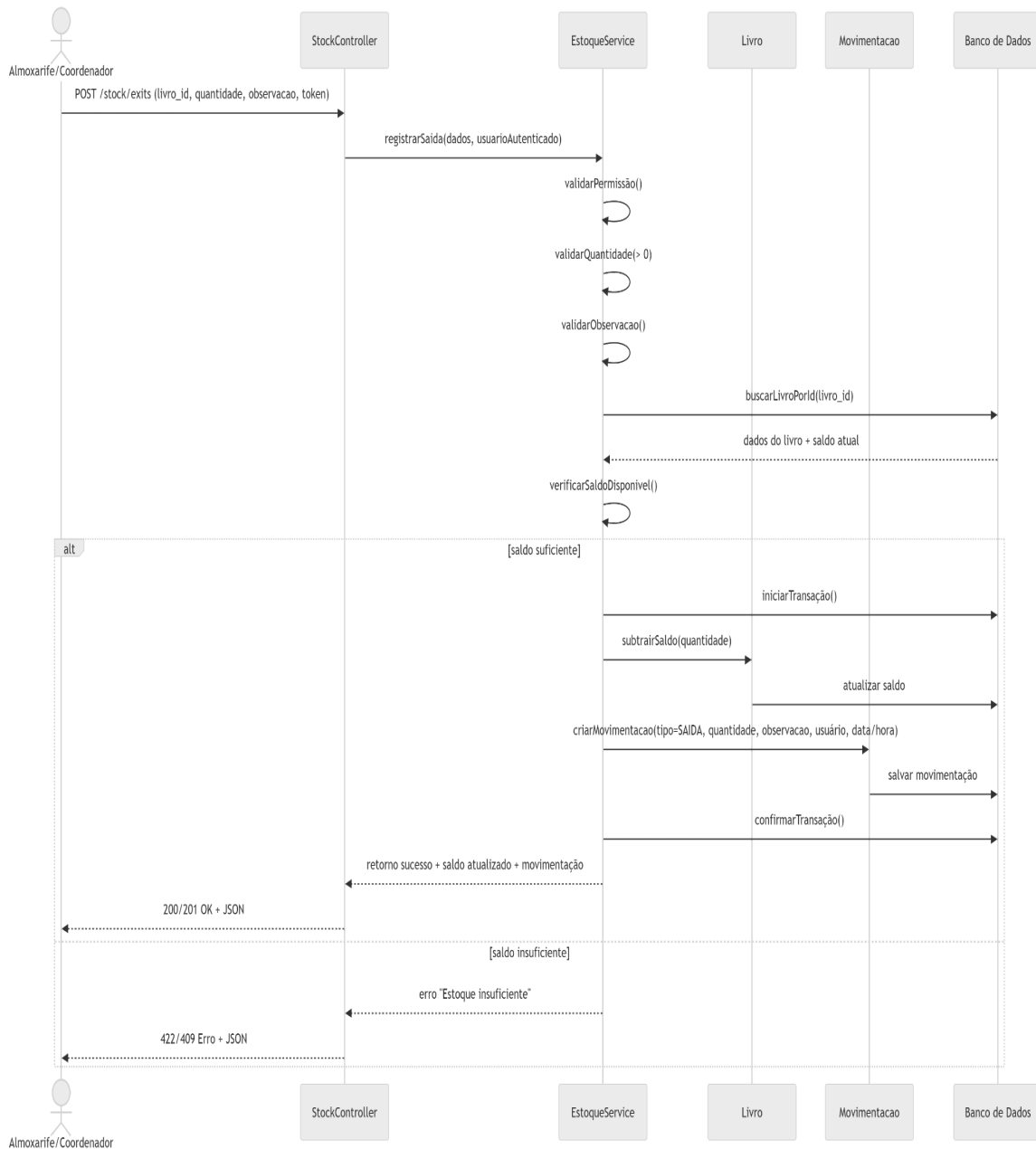
7. Diagrama de Classe

O diagrama é composto por cinco classes: Usuário, Livro, Movimentação, AuthService e Estoque Service. Usuário e Livro possuem relacionamento 1:N com Movimentação, indicando que um usuário pode realizar várias movimentações e um livro pode ter várias movimentações. AuthService gerencia autenticação (login, logout, token), enquanto Estoque Service concentra as regras de estoque (entrada, saída, validação de saldo e baixo estoque).



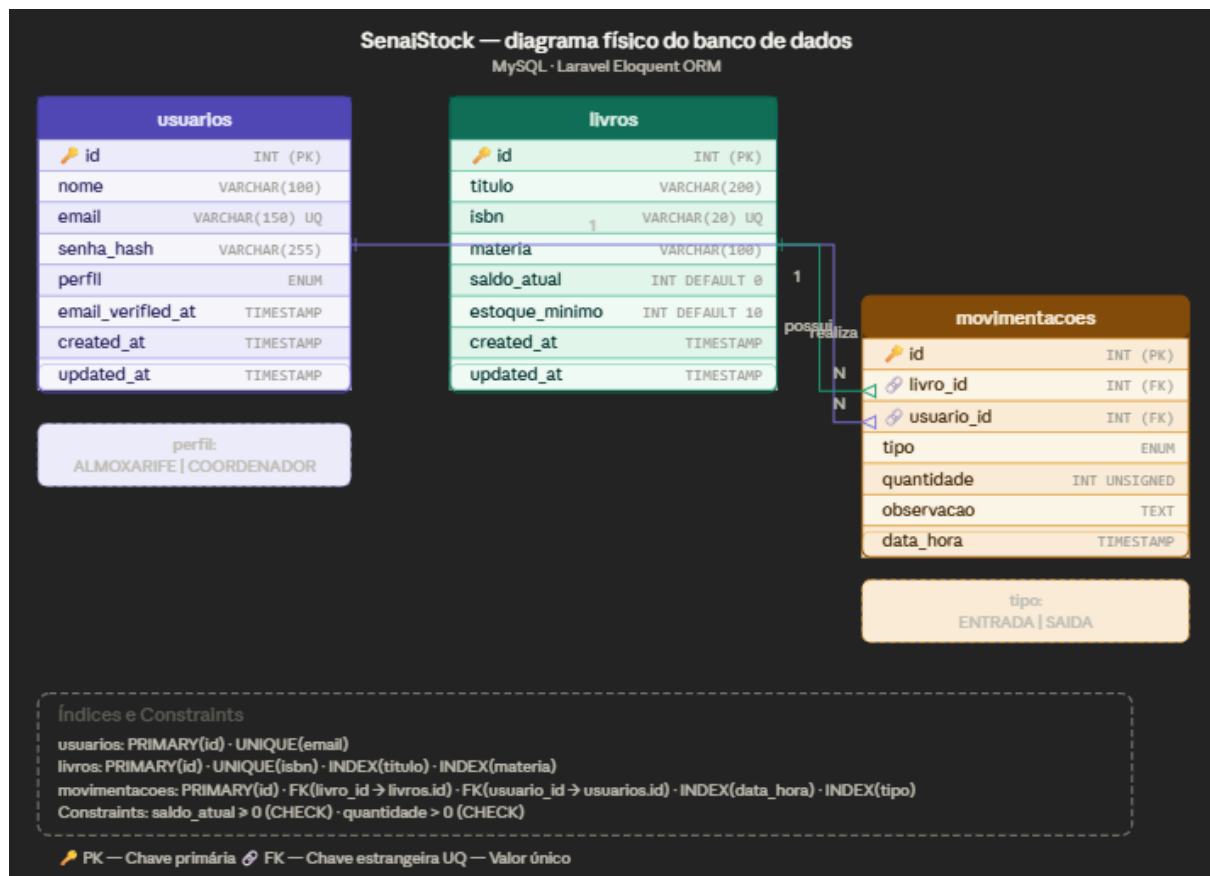
8. Diagrama de Sequência

O diagrama mostra um sistema onde o usuário primeiro se autentica no AuthService (login e geração de token). Após isso, ele utiliza o Estoque Service para registrar entradas e saídas de livros. Cada ação gera uma Movimentação, que atualiza o saldo do Livro e fica vinculada ao Usuário que realizou a operação.



9. Diagrama do Banco de Dados

O banco de dados do Senai Stock é composto por três tabelas: usuários, livros e movimentações. A tabela usuários armazena os dados de acesso com perfil (almoxarife ou coordenador). A tabela livros registra o catálogo com título, ISBN, matéria e saldo atual. A tabela de movimentações mantém o histórico de todas as entradas e saídas, incluindo livro, usuário, quantidade, tipo e observação. Os relacionamentos são de 1:N entre usuários e movimentações e entre livros e movimentações.



10. Metodologia de Desenvolvimento

10.1 Metodologia Ágil Utilizada

O projeto será desenvolvido utilizando **Scrum**, com divisão em Sprints previamente definidas pelo o docente.

O desenvolvimento será incremental, permitindo que a cada Sprint sejam realizadas melhorias, correções e validações, garantindo evolução contínua do sistema.

10.2 Organização das Sprints

O projeto está dividido em 6 Sprints, contemplando:

- Levantamento e refinamento de requisitos
- Modelagem (UML e Banco de Dados)
- Implementação da API
- Testes e validações
- Implantação
- Apresentação final

10.3 Ferramenta de Gestão

Para organização das tarefas e acompanhamento do progresso, foi utilizada a ferramenta **Trello**, onde foram criados quadros, listas e cartões correspondentes às Sprints e atividades do projeto.

Através do **GitHub** foi realizado o upload dos arquivos do projeto, como os códigos e as documentações

Toda a prototipação do projeto foi desenvolvido e construído usando o **Figma**

Link do Trello:

[Trello](#)

Link do GitHub:

[GitHub](#)

Link do Figma:

[Figma](#)